

Lecture 10: Introduction to Apache Spark

DSL351 / DSP352

Amit Kumar Dhar

ED1, 406A

amitkdhar@iitbhlai

January 28, 2026

Why Spark?

Limitations of MapReduce

- **Disk I/O Bottleneck:** MapReduce writes intermediate results to HDFS after every Map and Reduce phase.
- **Iterative Algorithms:** For algorithms like PageRank or K-Means, MapReduce reloads data from disk in every iteration.
- **High Latency:** Startup overhead of JVMs for every task.
- **Interactive Analysis:** Not suitable for ad-hoc queries due to high job overhead.

Visualizing MapReduce Overhead



- **Inefficiency:** Iterative ML algorithms can be 10x-100x slower.

The Spark Solution

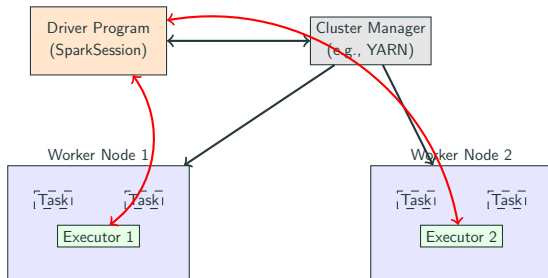
- **In-Memory Computing:** Spark keeps data in RAM between steps, avoiding costly disk I/O.
- **Fault Tolerance:** Instead of replication, Spark uses **Lineage** to recompute lost data.
- **Unified Engine:** One framework for Batch, SQL, Streaming, ML, and Graph.
- **Productivity:** Rich APIs in Scala, Java, Python, and R.

Spark Architecture

Master-Slave Architecture

- **Driver:** The orchestrator. Hosts the SparkSession and schedules tasks.
- **Executors:** The workers. Run tasks and store data for the application.
- **Cluster Manager:** Allocates resources across applications (YARN, Mesos, K8s).

Spark Architecture Diagram



The Driver Program

- Entry point of the Spark application.
- Converts user code into a **DAG** (Directed Acyclic Graph).
- Breaks DAG into **Stages** and **Tasks**.
- Distributes tasks to Executors.
- Collects results and presents them to the user.

- **Isolation:** Each application has its own executors.
- **Persistence:** Executors store data partitions in memory (RAM) or disk.
- **Slots:** Units of parallel execution. Usually one slot = one thread = one vCore.
- If an Executor has 4 slots, it can run 4 tasks simultaneously.

Resilient Distributed Datasets (RDDs)

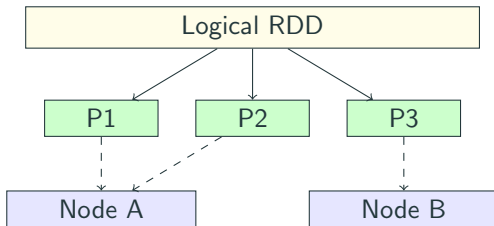
What is an RDD?

Resilient Distributed Dataset (RDD) is the core abstraction.

- **Resilient:** Fault-tolerant. Can be reconstructed from lineage if a partition is lost.
- **Distributed:** Partitioned across multiple nodes.
- **Dataset:** A collection of Java/Scala objects or Python tuples.

- **Immutable:** Once created, it cannot be changed. Transformations produce new RDDs.
- **Partitioned:** Logical chunks of data.
- **Lazy Evaluation:** Computation only happens when an **Action** is called.
- **Lineage:** Keeps track of the operations applied to the original data.

RDD Partitioning Strategy



- Number of partitions = Level of parallelism.

Transformations vs. Actions

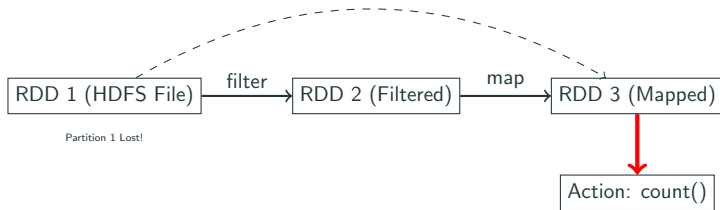
Transformations (Lazy):

- Create new RDD from existing.
- Build DAG lineage.
- Narrow: `map`, `filter`.
- Wide: `reduceByKey`, `join`.

Actions (Eager):

- Return value to Driver or write to Disk.
- Trigger DAG execution.
- `count`, `collect`, `take`, `saveAsTextFile`.

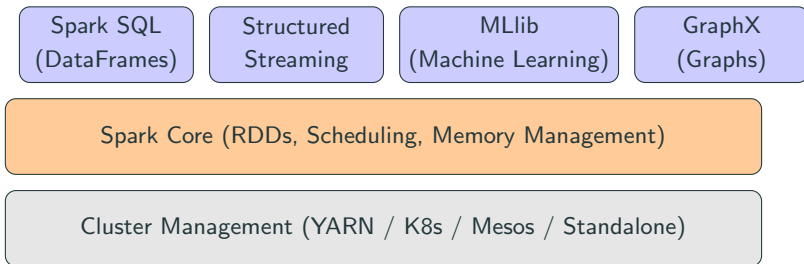
Lineage Graph (DAG)



- Fault tolerance without replication. Lost parts are recomputed.

Spark Ecosystem

The Unified Analytics Engine



- Module for structured data processing.
- **DataFrames:** RDDs with schema (Columns and Types).
- **Catalyst Optimizer:** Automatically optimizes logical and physical plans.
- Query data using SQL or the Dataset API.

- Scalable, high-throughput, fault-tolerant stream processing.
- **Micro-batching:** Data treated as a series of small batch jobs.
- **Structured Streaming:** Built on the SQL engine, offering exactly-once guarantees.

- Spark's machine learning library.
- Contains high-quality algorithms (Classification, Regression, Clustering).
- **Pipelines:** Tools for constructing, evaluating, and tuning ML workflows.

- API for graphs and graph-parallel computation.
- Unifies ETL, exploratory analysis, and iterative graph computation in one system.
- Examples: PageRank, Connected Components.

Code Examples

Word Count in PySpark (RDD API)

```
1 from pyspark import SparkContext
2
3 sc = SparkContext("local", "WordCount")
4
5 # Read file
6 lines = sc.textFile("hdfs:///data/text.txt")
7
8 # Transformations
9 counts = lines.flatMap(lambda line: line.split(" ")) \
10                  .map(lambda word: (word, 1)) \
11                  .reduceByKey(lambda a, b: a + b)
12
13 # Action
14 print(counts.collect())
```


Modern Spark: SparkSession & DataFrames

```
1 from pyspark.sql import SparkSession
2
3 spark = SparkSession.builder \
4     .appName("DataFrameExample") \
5     .getOrCreate()
6
7 # Load JSON
8 df = spark.read.json("employees.json")
9
10 # DSL Style
11 df.filter(df["age"] > 30).select("name").show()
12
13 # SQL Style
14 df.createOrReplaceTempView("emp")
15 spark.sql("SELECT name FROM emp WHERE age > 30").show()
```

Questions?